

ANALIZA I INTEGRACIJA KREACIJSKIH DIZAJNERSKIH PATERNA

Uvod i kontekst postojećeg sistema

Sistem modelira ključne domenske entitete kao što su Korisnik, Rezervacija, ParkingObjekat, Transakcija i Izvjestaj. Sistem koristi Strukturne paterne (ParkingReservationFacade koji pojednostavljuje interakciju sa podsistemima i PaymentInterface sa adapterima StripeAdapter i PayPalAdapter koji izoluju eksterne servise za plaćanje).

Trenutna struktura se oslanja na direktno vezivanje (tight coupling) i prebacivanje prevelike odgovornosti na klijentske klase, što narušava Single Responsibility Principle (SRP) i Open/Closed Principle (OCP).

Potrebe za kreacijskim paternima

Kreacijski dizajnerski paterni (Creational Design Patterns) bave se mehanizmima kreiranja objekata, stvarajući objekte na način koji je prilagođen situaciji. Bez njihove primjene, sistem se suočava sa sljedećim problemima:

- **Direktno instanciranje (ključna riječ new):** Kada klasa Korisnik ili fasada direktno pozivaju konstruktore drugih klasa, postaju zavisne o konkretnim implementacijama, a ne apstrakcijama.
- **Kompleksnost inicijalizacije:** Entiteti poput Izvjestaj ili Rezervacija zahtijevaju postavljanje većeg broja parametara. Logika validacije i sklapanja unutar klijentske klase otežava održavanje i povećava podložnost greškama.
- **Nemogućnost proširenja:** Uvođenje novog tipa izvještaja ili korisnika zahtijeva izmjenu postojećih klasa, što direktno krši OCP.

Odabir i primjena kreacijskih paterna

Patern 1: Factory Method

- **Lokacija problema:** Klasa Korisnik trenutno sadrži metodu sa stereotipom <<Create>> `Korisnik(...)` i direktno upravlja kreiranjem instanci. Sistem generiše različite vrste izvještaja (Izvjestaj) koji zavise od enumeracije `FormatIzvjestaja` (PDF, EXCEL, WORD).

- **Rješenje i opravdanje:** Uvođenjem Factory Method paterna za kreiranje izvještaja izolujemo logiku instanciranja. Umjesto da klijent (npr. administrator ili fasada) direktno instancira konkretni izvještaj, kreira se apstraktni interfejs `IzvjestajFactory` sa metodom `KreirajIzvjestaj()`, koju nasljeđuju konkretne fabrike: `PDFIzvjestajFactory` i `ExcelIzvjestajFactory`. Ovo omogućava dodavanje novih formata u budućnosti bez izmjene postojećeg klijentskog koda.
- **Prednosti u kontekstu sistema:**
 - Potpuno izoluje logiku kreiranja izvještaja od poslovne logike.
 - Dodavanje `WordIzvjestajFactory` ne zahtijeva nikakvu izmjenu postojećeg koda.
 - Omogućava unit testiranje svake fabrike zasebno.

Patern 2: Builder

- **Lokacija problema:** Klasa `Rezervacija` je centralni i najkompleksniji entitet sistema. Sadrži mnogo atributa: `rezervacijaID`, `vrijemePocetka`, `vrijemeIsteka`, `QRCode`, `status`, `vlasnikID` i `parkingID`. Kreiranje rezervacije zahtijeva korak-po-korak provjeru, generisanje QR koda, kalkulaciju vremena i vezivanje za korisnika i parking objekat.
- **Rješenje i opravdanje:** Builder patern razdvaja konstrukciju složenog objekta od njegove reprezentacije. Uvođenjem klase `RezervacijaBuilder`, omogućavamo kreiranje objekta `Rezervacija` kroz jasan i čitljiv *fluent interface*. Fasada `ParkingReservationFacade` tada koristi ovaj Builder kako bi bezbjedno i validirano sklopila objekat rezervacije, smanjujući kompleksnost konstruktora i sprečavajući slanje pogrešnih parametara.
- **Prednosti u kontekstu sistema:**
 - Eliminuje tzv. *telescoping constructor* antipatern (previše parametara u konstrukturu).
 - Centralizuje validaciju – nemoguće je kreirati nevažeću rezervaciju.
 - *Fluent interface* čini kôd čitljivijim i manje podložnim greškama.

Patern 3: Abstract Factory

- **Lokacija problema:** Sistem podržava više načina plaćanja (Stripe, PayPal). Svaki provajder zahtijeva vlastiti skup objekata: `PaymentSession`, `PaymentValidator` i `PaymentReceipt`. Direktno instanciranje ovih objekata po tipu provajdera dovodi do dupliranja logike i narušavanja OCP principa svaki put kada se doda novi provajder plaćanja.
- **Rješenje i opravdanje:** Abstract Factory pruža interfejs za kreiranje porodice srodnih objekata bez specificiranja konkretnih klasa. Uvođenjem `IPaymentFactory` interfejsa koji kreira čitav skup payment objekata, svaki provajder implementira svoju fabriku (`StripePaymentFactory`, `PayPalPaymentFactory`). Klijentski kôd (fasada ili controller) koristi samo `IPaymentFactory` i ostaje neizmjenjen pri dodavanju novog provajdera.

- **Prednosti u kontekstu sistema:**
 - Garantuje konzistentnost - objekti iste fabrike su uvijek kompatibilni međusobno.
 - Dodavanje novog provajdera (npr. `KlikPayFactory`) ne mijenja postojeći kôd.
 - Integrira se prirodno s postojećim `PaymentInterface` adapterima.

Patern 4: Singleton (Jedinstven primjerak)

- **Lokacija problema:** Sistem koristi konfiguraciju (connection string, SMTP podešavanja, API ključeve) koja se učitava iz `appsettings.json`. Ako bi svaki kontroler ili servis kreirao vlastitu instancu konfiguracijske klase, riskiramo nekonzistentnost podataka, višestruko čitanje fajlova i nepotrebno trošenje resursa.
- **Rješenje i opravdanje:** Singleton patern osigurava da klasa ima samo jednu instancu i pruža globalnu tačku pristupa toj instanci. In kontekstu `ParkirajBa` sistema, klasa `KonfiguracijaServisa` (koja čuva podešavanja aplikacije) i `NotifikacijaBroker` (koji upravlja slanjem emailova i push notifikacija) su idealni kandidati. U ASP.NET Core, Singleton se najčešće implementira kroz DI container registracijom sa `AddSingleton`.
- **Prednosti u kontekstu sistema:**
 - Jedan SMTP klijent za cijelu aplikaciju – nema višestrukih konekcija.
 - `Lazy<T>` osigurava thread-safe inicijalizaciju bez eksplicitnog locking-a.
 - U ASP.NET Core, DI container automatski upravlja životnim ciklusom singletona.

Patern 5: Prototype

- **Lokacija problema:** U sistemu se često kreiraju slični parking objekti ili predlošci rezervacija s minimalnim razlikama (npr. sezonski parking, VIP parking). Svaki put kreiranje novog objekta od nule je skupo jer uključuje inicijalizaciju, postavljanje default vrijednosti i eventualno dohvaćanje podataka iz baze.
- **Rješenje i opravdanje:** Prototype patern omogućava kloniranje postojećih objekata umjesto kreiranja novih od nule. Svaki `ParkingObjekat` ili `Pricing` može implementirati metodu `Kloniraj()` koja vraća kopiju s istim atributima. Operator tada može klonirati postojeći parking predložak i samo modificirati specifične attribute (npr. naziv, lokaciju, cijenu) bez ponovne inicijalizacije.
- **Prednosti u kontekstu sistema:**
 - Eliminuje skupe inicijalizacijske sekvence za slične objekte.
 - Operator može brzo kreirati varijante parkinga na osnovu template-a.
 - Kombinuje se dobro s Builder paternim – Builder može klonirati, pa refinirati.

Patern 6: Object Pool (Bazen objekata)

- **Lokacija problema:** Slanje emailova (email potvrde rezervacije, QR kod) koristi `SmtplibClient` koji se kreira i uništava pri svakom pozivu metode `PosaljiEmail`. Kreiranje

SMTP konekcije je skupa operacija, a pri povećanom prometu ovo može postati usko grlo sistema.

- **Rješenje i opravdanje:** Object Pool patern održava skup unaprijed inicijaliziranih objekata koji se posuđuju i vraćaju, umjesto da se kreiraju i uništavaju po potrebi. U ASP.NET Core kontekstu, Microsoft nudi `ObjectPool<T>` implementaciju. Umjesto instanciranja `SmtpClient` unutar svake metode, kreira se pool klijenata koji se reupotrebljavaju, što dramatično smanjuje latenciju i opterećenje sistema.

Prednosti u kontekstu sistema:

- Eliminuje overhead kreiranja SMTP konekcije pri svakom emailu.
- Automatski upravlja brojem aktivnih konekcija.
- Lako se proširuje na pooling DB konekcija ili QRCode generatora.

Svi paterni se prirodno integrišu u ASP.NET Core arhitekturu kroz DI kontejner, repository pattern i postojeću fasadnu strukturu sistema, bez narušavanja trenutnog dizajna.